

UNIT - II

- Searching- Searching For Solutions,
- Uniformed Search Strategies – Breadth First Search, Depth First Search.
- Search With Partial Information (Heuristic Search) Hill Climbing, A* ,AO* Algorithms,
- Problem Reduction,
- Game Playing-Adversial Search,
- Games,
- Mini-Max Algorithm.

Why Unit–II Is Important for AI

Unit–II provides the **problem-solving foundation of Artificial Intelligence**. It moves students from:

Basic exploration → intelligent guided search → problem decomposition → competitive decision-making.

This is why Unit–II connects directly with learning **methods of solving problems using AI**.

SEARCHING



Searching for Solutions

Find a path from the initial state to the goal state.



How can we explore states?

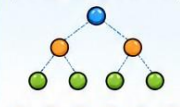
Use search strategies to explore possible states.



Uninformed Search

No knowledge about the goal.

BFS (Breadth First Search)



Explore level by level
(uses Queue)

DFS (Depth First Search)



Go deep into a branch
(uses Stack)



Heuristic Search

Uses knowledge to find better paths.

Hill Climbing



Move to a better
neighbor step
by step.

A*



Considers both
cost so far and
estimated cost
to goal.

AO*



Works on AND-OR
graphs for complex
problems.



Problem Reduction

Break a complex problem into smaller subproblems.



Game Playing

Make decisions in a game to achieve the goal.



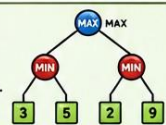
Adversarial Search

An opponent tries to minimize our success.



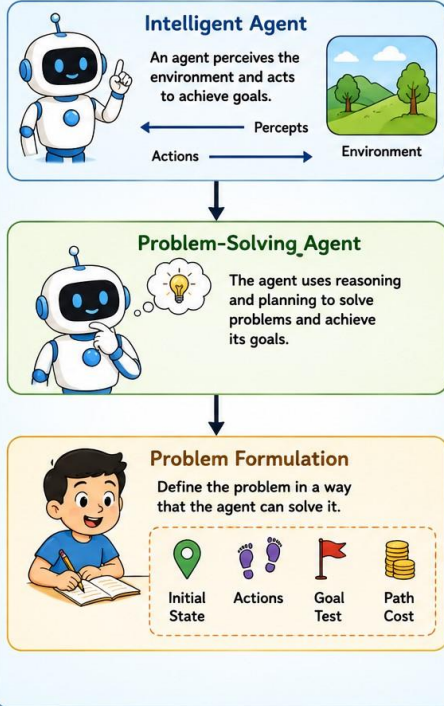
Minimax

Choose the best move by considering opponent's best move.



UNIT - I

Foundation: Understanding the Problem

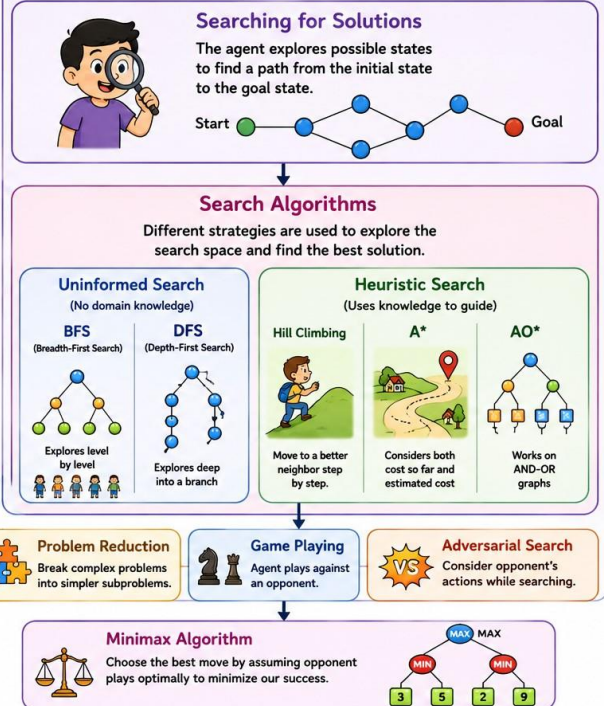


Once the problem is formulated, the agent searches for a solution.



UNIT - II

Solving the Problem: Searching for Solutions



Search Algorithms in AI

- Search algorithms in AI are techniques used to find solutions by exploring possible paths or states in a problem space. They help intelligent systems make decisions, solve problems and reach goals efficiently in applications such as navigation, robotics, game playing and path finding.
- Helps AI systems determine the best possible solution among multiple choices
- Widely used in decision-making, optimization and route-planning problems

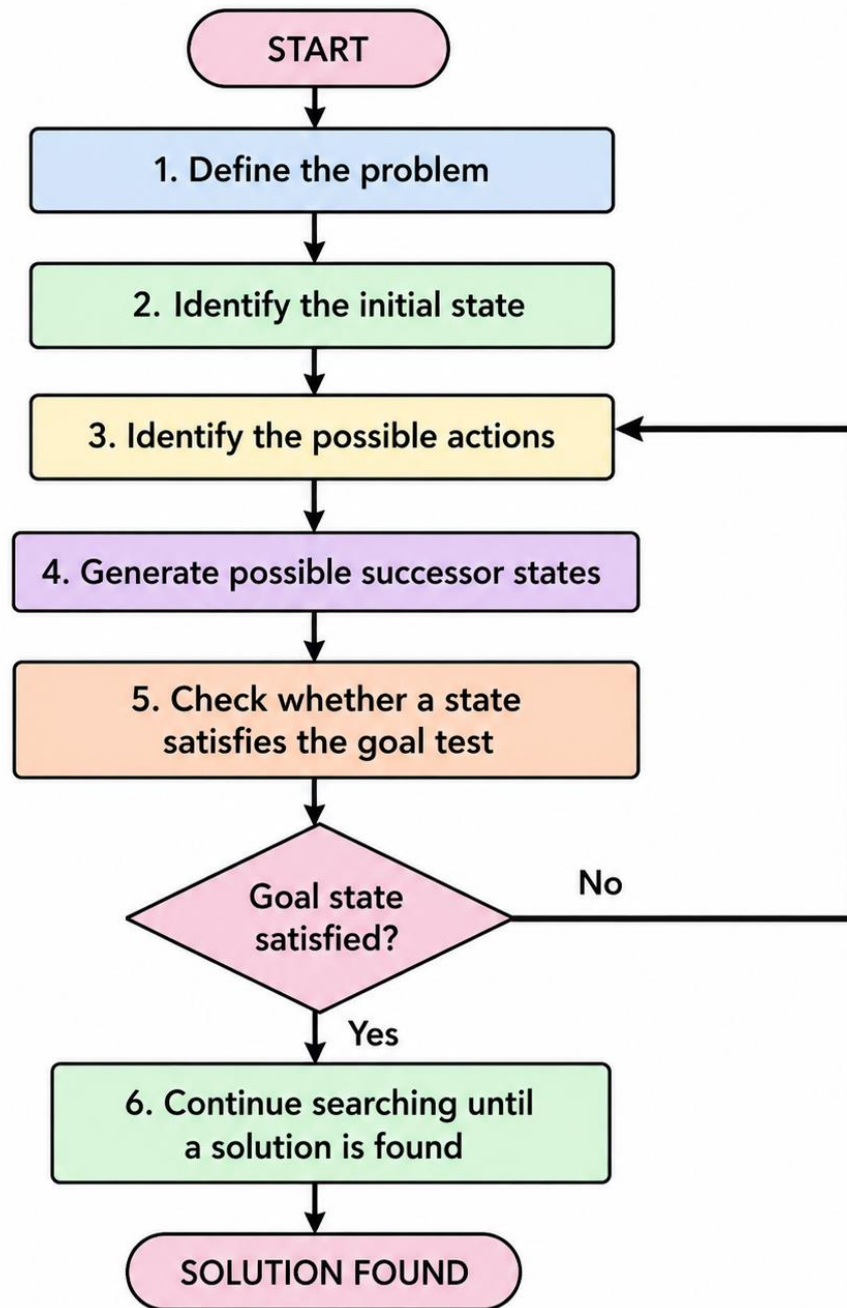
Basic idea:

Problem → Search → Solution

Searching for Solutions

An AI agent generally performs the following steps:

1. **Define the problem**
2. Identify the **initial state**
3. Identify the **possible actions**
4. Generate possible **successor states**
5. Check whether a state satisfies the **goal test**
6. Continue searching until a solution is found



Types of Search Algorithms

There are mainly 2 types of search algorithms i.e

- Uninformed Search Algorithms and
- Informed Search Algorithms.

Uninformed Search Algorithms

Unit-II: Uninformed Search Strategies (BFS, DFS)

Definition: Uninformed Search, also called Blind Search, explores possible states without using heuristic information about which state is closer to the goal.

1. Breadth First Search (BFS)

Definition: BFS explores the search tree level by level. It first visits all nodes at the current depth and then moves to the next depth.

Example:

```
A
/ \
B C
/ \ / \
D E F G
```

BFS order: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G$

Data Structure: Queue

FIFO = First In, First Out

Real-world applications

- Web crawling
- Finding shortest paths in unweighted networks
- Social-network connection levels
- Network broadcasting

Web Crawler Example – College Website

A college website can be represented as a graph where webpages are nodes and hyperlinks are connections.

```
Home
■■■ About
■■■ Academics
■■■ Admissions
    ■■■ Vision
    ■■■ Courses
    ■■■ Eligibility
```

BFS: $\text{Home} \rightarrow \text{About} \rightarrow \text{Academics} \rightarrow \text{Admissions} \rightarrow \text{Vision} \rightarrow \text{Courses} \rightarrow \text{Eligibility}$

Idea: BFS searches broadly before going deeper.

2. Depth First Search (DFS)

Definition: DFS explores one branch as deeply as possible before backtracking and exploring another branch.

Example:

```
A
/ \
B C
/ \ / \
D E F G
```

One possible DFS order: $A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow F \rightarrow G$

Data Structure: Stack

LIFO = Last In, First Out

Real-world applications

- Maze-solving robot
- Exploring folders in a computer
- Backtracking problems
- Exploring paths in a graph

Maze-Solving Robot Example

A robot can follow one path deeply. If it reaches a dead end, it backtracks and tries another path.

```
START
↓
Path A
↓
Path B
↓
Dead End
↓
BACKTRACK
↓
Path C
↓
GOAL
```

Idea: DFS searches deep along one path and backtracks when necessary.

3. BFS vs DFS

Feature	BFS	DFS
Full form	Breadth First Search	Depth First Search
Search method	Level by level	Deep into one branch
Data structure	Queue	Stack
Principle	FIFO	LIFO
Main operation	Explore broadly	Explore deeply
Backtracking	Less prominent	Important
Example	Web crawler	Maze-solving robot

4. Easy Way to Remember

UNINFORMED SEARCH

■■■ BFS → Queue → FIFO → Level by Level → Web Crawler

■■■ DFS → Stack → LIFO → Deep First → Maze Robot

5. Classroom / Whiteboard Activity

Draw a search tree on the whiteboard. Ask students to perform BFS using a queue and DFS using a stack. For the virtual-lab activity, represent a college website as webpages connected by hyperlinks and demonstrate how the crawler visits pages.

Student-friendly conclusion: BFS searches wide and level-by-level, whereas DFS searches deep along one path and backtracks when necessary.

Heuristic Search Techniques in AI

Heuristic search techniques are used **for problem-solving in AI systems**. These techniques help find the most efficient path from a starting point to a goal, making them essential for applications such as navigation systems, game playing, and optimization problems.

- Heuristic search operates within the search space of a problem to find the best or near-optimal solution using systematic algorithms.
- Unlike brute-force methods, which exhaustively evaluate all possible solutions, heuristic search leverages heuristic information to guide the search toward more promising paths.

In this context, **heuristics** refer to a set of criteria or rules of thumb that provide an estimate of the most viable solution. By balancing **exploration** (searching new possibilities) and **exploitation** (refining known solutions), heuristic algorithms efficiently solve complex problems that would otherwise be computationally expensive.

Significance of Heuristic Search in AI

The advantage of heuristic search techniques in AI is their ability to efficiently navigate large search spaces. By prioritizing the most promising paths, heuristics significantly reduce the number of possibilities that need to be explored. This not only accelerates the search process but also enables AI systems to solve complex problems that would be impractical for exact algorithms.

Components of Heuristic Search

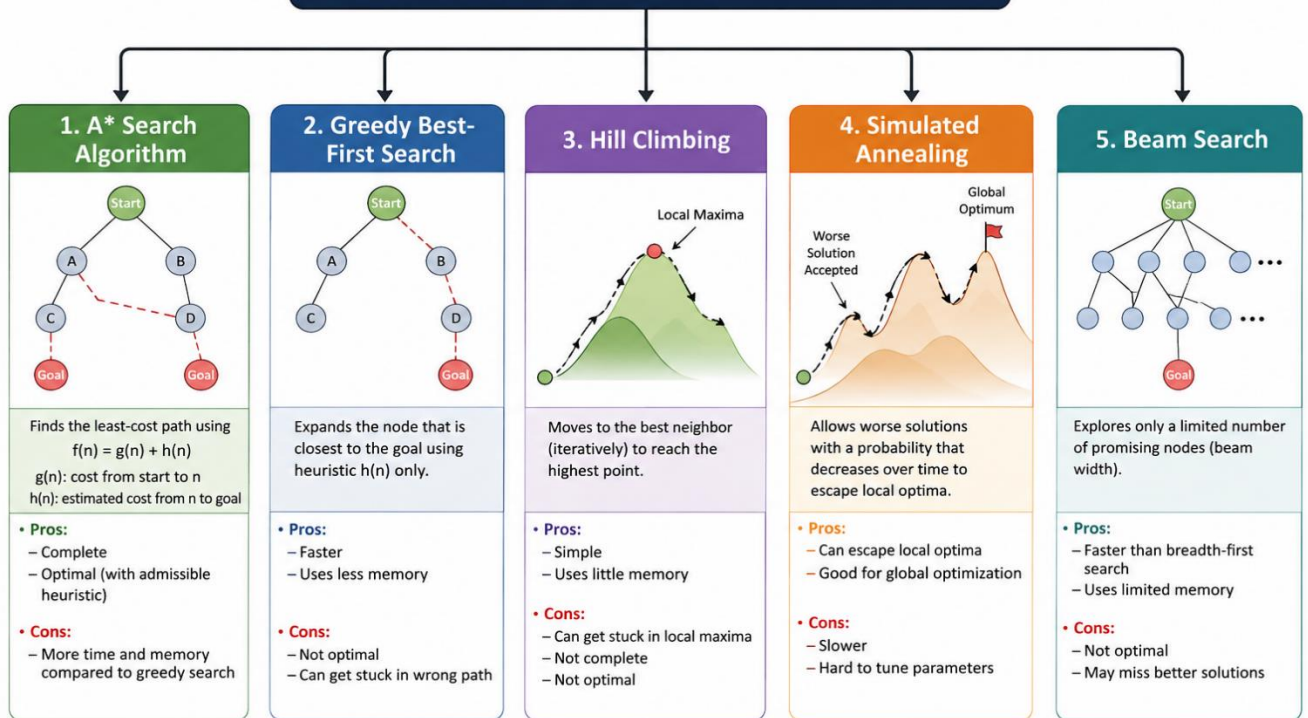
Heuristic search algorithms typically comprise several essential components:

1. **State Space:** This implies that the totality of all possible states or settings, which is considered to be the solution for the given problem.
2. **Initial State:** The instance in the search tree of the highest level with no null values, serving as the initial state of the problem at hand.
3. **Goal Test:** The exploration phase ensures whether the present state is a terminal or consenting state in which the problem is solved.
4. **Successor Function:** This creates a situation where individual states supplant the current state which represent the possible moves or solutions in the problem space.
5. **Heuristic Function:** The function of a heuristic is to estimate the value or distance from a given state to the target state. It helps to focus the process on regions or states that have the prospect of achieving the goal.

Types of Heuristic Search Techniques

Over the history of heuristic search algorithms, there have been a lot of techniques created to improve them further and attend different problem domains. Some prominent techniques include:

TYPES OF HEURISTIC SEARCH



Note: The performance of heuristic search algorithms depends on the choice of heuristic function and problem characteristics.

Advantages of Heuristic Search Techniques

Heuristic search techniques offer several advantages:

- **Efficiency:** By focusing on the most promising paths, heuristic search significantly reduces the number of possibilities explored, saving both time and computational resources.
- **Optimality:** When using **admissible heuristics**, certain algorithms like A* can guarantee an optimal solution, ensuring the best possible outcome.
- **Versatility:** Heuristic methods are adaptable and can be applied to a wide range of problems, from **pathfinding and optimization to game AI and robotics**.

Limitations of Heuristic Search Techniques

Despite their advantages, heuristic search techniques also have some limitations:

- **Heuristic Quality:** The effectiveness of heuristic search heavily depends on the quality of the heuristic function. Poorly designed heuristics can lead to inefficient or suboptimal solutions.

- **Space Complexity:** Some heuristic algorithms require large amounts of memory, especially when dealing with extensive search spaces, making them less practical for resource-limited environments.
- **Domain-Specificity:** Designing effective heuristics often requires domain-specific knowledge, which can make it difficult to create general-purpose heuristic approaches.

Hill Climbing in Artificial Intelligence

Hill climbing is a heuristic search algorithm that belongs to the family of local search methods. It is designed to solve problems where the goal is to find an optimal (or near-optimal) solution by iteratively moving from the current state to a better neighboring state, according to a heuristic or evaluation function.

- It is an optimisation technique used in artificial intelligence (AI) to find solutions for a wide variety of problems.
- It operates on the principle of incrementally improving a solution by making local changes and evaluating their merit.
- Its simplicity, intuitive logic and adaptability to different problems make it a go-to method.

Hill Climbing Algorithms

Hill climbing follows these steps:

1. **Initial State:** Start with an arbitrary or random solution (initial state).
2. **Neighboring States:** Identify neighboring states of the current solution by making small adjustments (mutations or tweaks).
3. **Move to Neighbor:** If one of the neighboring states offers a better solution (according to some evaluation function), move to this new state.

4. **Termination:** Repeat this process until no neighboring state is better than the current one. At this point, we have reached a local maximum or minimum.

Features of Hill Climbing Algorithm

1. Variant of Generating and Testing Algorithm: Hill Climbing is a specific variant of the generating and testing algorithms. The process involves: This iterative feedback mechanism allows Hill Climbing to refine its search by using information from previous evaluations to inform future moves in the search space.

- **Generating possible solutions:** The algorithm creates potential solutions within the search space.
- **Testing solutions:** Each generated solution is evaluated to determine if it meets the desired criteria.
- **Iteration:** If a satisfactory solution is found, the algorithm terminates; otherwise, it returns to the generation step.

2. Greedy Approach: Hill Climbing algorithm uses greedy approach, meaning that at each step, it moves in the direction that optimizes the objective function. This strategy aims to find the optimal solution efficiently by making the best immediate choice without considering the overall problem context.

Types of Hill Climbing in Artificial Intelligence

1. Simple Hill Climbing Algorithm: Simple Hill Climbing is a straightforward variant of hill climbing where the algorithm evaluates each neighbouring node

one by one and selects the first node that offers an improvement over the current one.

2. Steepest-Ascent Hill Climbing: Steepest-Ascent Hill Climbing is an enhanced version of simple hill climbing. Instead of moving to the first neighboring node that improves the state, it evaluates all neighbors and moves to the one offering the highest improvement (steepest ascent).

3. Stochastic Hill Climbing: Stochastic Hill Climbing introduces randomness into the search process. Instead of evaluating all neighbors or selecting the first improvement, it selects a random neighboring node and decides whether to move based on its improvement over the current state.

State-Space Diagram in Hill Climbing

The state-space diagram is a visual representation of all possible states the search algorithm can reach, plotted against the values of the objective function (the function we aim to maximize).

in the state-space diagram:

- **X-axis:** Represents the state space which includes all the possible states or configurations that the algorithm can reach.
- **Y-axis:** Represents the values of the objective function corresponding to each state.

The optimal solution in the state-space diagram is represented by the state where the **objective function** reaches its maximum value, also known as the **global maximum**.

Key Regions in the State-Space Diagram

1. **Local Maximum:** A local maximum is a state better than its neighbors but not the best overall. While its objective function value is higher than nearby states, a global maximum may still exist.
2. **Global Maximum:** The global maximum is the best state in the state-space diagram where the objective function achieves its highest value. This is the optimal solution the algorithm seeks.
3. **Plateau/Flat Local Maximum:** A plateau is a flat region where neighboring states have the same objective function value, making it difficult for the algorithm to decide on the best direction to move.
4. **Ridge:** A ridge is a higher region with a slope which can look like a peak. This may cause the algorithm to stop prematurely, missing better solutions nearby.
5. **Current State:** The current state refers to the algorithm's position in the state-space diagram during its search for the optimal solution.
6. **Shoulder:** A shoulder is a plateau with an uphill edge allowing the algorithm to move toward better solutions if it continues searching beyond the plateau.

Advantages of Hill Climbing Algorithm

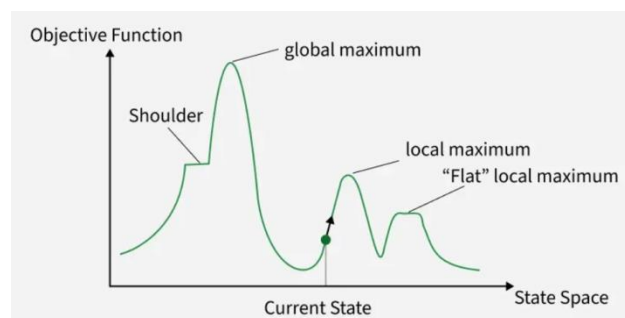
1. **Simplicity and Ease of Implementation:** Hill Climbing is a simple and intuitive algorithm that is easy to understand and implement making it accessible for developers and researchers.
2. **Versatility:** The algorithm can be applied to a wide variety of optimization problems, including those with large search spaces and complex constraints.
3. **Efficiency in Finding Local Optima:** Hill Climbing is often highly efficient at finding local optima making it a suitable choice for problems where a good solution is required quickly.
4. **Customizability:** The algorithm can be easily modified or extended to incorporate additional heuristics or constraints allowing for more tailored optimization approaches.

Applications of Hill Climbing in AI

- **Pathfinding:** It is used in AI systems that need to navigate or find the shortest path between points such as in robotics or game development.
- **Optimization:** It can be used for solving optimization problems where the goal is to maximize

or minimize a particular objective function such as scheduling or resource allocation problems.

- **Game AI:** In certain games, AI uses hill climbing to evaluate and improve its position relative to an opponent's.
- **Machine Learning:** It is sometimes used for hyperparameter tuning where the algorithm iterates over different sets of hyperparameters to find the best configuration for a machine learning model.



1. A* Search Algorithm

A* (A-star) is a **heuristic search algorithm** used to find the least-cost path from an initial state to a goal state.

Heuristic search is an informed search strategy that uses a heuristic function to estimate how close a node is to the goal and guides the search toward the most promising node.

It combines:

- **Actual cost** from the start node $\rightarrow g(n)$
- **Estimated cost** from node n to the goal $\rightarrow h(n)$

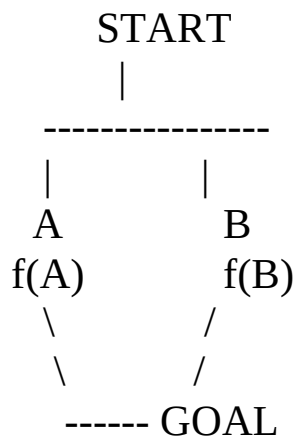
The evaluation function is:

$$f(n) = g(n) + h(n)$$

Where:

Term	Meaning
$g(n)$	Actual cost from start node to n
$h(n)$	Heuristic/estimated cost from n to goal
$f(n)$	Estimated total cost of solution through n

Basic idea



A* always selects the **OPEN node having the smallest $f(n)$ value.**

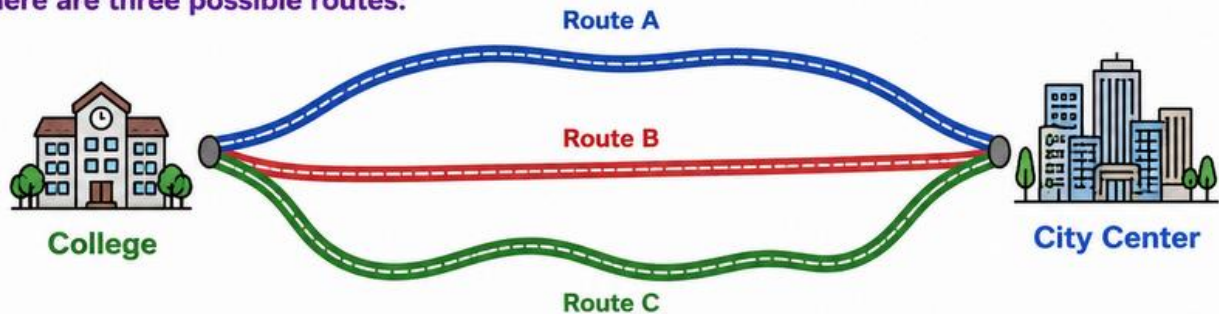
Steps of A*

1. Put the initial node in the **OPEN list**.
2. Calculate $f(n) = g(n) + h(n)$.
3. Select the node with the **lowest $f(n)$** .
4. If it is the goal, stop.
5. Otherwise, expand the node.
6. Generate its successors.
7. Calculate g , h , and f for each successor.
8. Update OPEN and CLOSED lists.
9. Repeat until the goal is reached.

A* Example – Route Finding (Google Maps / GPS)

Suppose you want to travel: **College** → **City Center**

There are three possible routes:



A* considers:

- $g(n)$: Distance already travelled from the start (College)
- $h(n)$: Estimated distance remaining to the destination (City Center)
- $f(n)$: Total estimated distance → $f(n) = g(n) + h(n)$

$g(n)$ = Distance already travelled

$h(n)$ = Estimated distance remaining

$f(n)$ = Total estimated distance

For example:

Route	$g(n)$ – Distance already travelled	$h(n)$ – Estimated remaining distance	$f(n) = g(n) + h(n)$ – Total estimated distance
A	5 km	3 km	8 km
B	4 km	6 km	10 km
C	6 km	1 km	7 km



A* selects the route with $\min f(n)$. Therefore, **Route C** is selected at this stage.

A (A-star) *algorithm** is a widely used **path-finding algorithm** in computer science, known for its efficiency and effectiveness in **route planning** for applications such as **maps, robotics, and games**.

Game	How A* can be used
Age of Empires	Units find paths around obstacles and terrain
StarCraft	Units navigate around buildings and other units
Warcraft	Characters and units find routes through the game map
Diablo	Characters and monsters navigate dungeon environments
Minecraft	Mobs can use pathfinding to navigate toward targets
Civilization	Units find routes across the map
The Elder Scrolls	NPCs navigate complex environments
Pac-Man	Ghost movement can be modeled using graph/pathfinding techniques
Tower Defense games	Enemies find paths from spawn points to the base
RPG games	NPCs and enemies navigate maps and dungeons

Game	Image	How A* can be used
Age of Empires		Units find paths around obstacles and terrain.
StarCraft		Units navigate around buildings and other units.
Warcraft		Characters and units find routes through the game map.
Diablo		Characters and monsters navigate dungeon environments.
Minecraft		Mobs can use pathfinding to navigate toward targets.
Civilization		Units find routes across the map.
The Elder Scrolls		NPCs navigate complex environments.
Pac-Man		Ghost movement can be modeled using graph/pathfinding techniques.
Tower Defense games		Enemies find paths from spawn points to the base.
RPG games		NPCs and enemies navigate maps and dungeons.

2.AO* Algorithm

AO* (**AND-OR Star**) is a heuristic search algorithm used to find an optimal solution in an **AND-OR graph**. It is an extension of the A* algorithm.

1. What is an AND-OR Graph?

An **OR node** represents alternative choices:



To solve **A**, we can choose either **B OR C**.

An **AND node** represents a problem that requires multiple sub-problems to be solved:



To solve **A**, we must solve **B AND C**.

2. Cost Calculation

For an **OR node**:

$$f(n) = \min[\text{cost}(n, n_i) + f(n_i)]$$

The algorithm selects the child with the **minimum estimated cost**.

For an **AND node**:

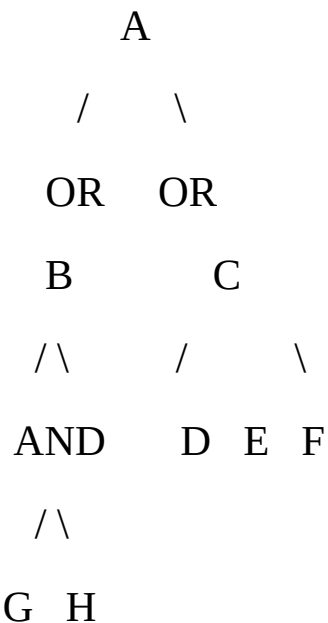
$$f(n) = \sum[\text{cost}(n, n_i) + f(n_i)]$$

The costs of **all required child nodes** are added.

3. AO* Algorithm Steps

1. Start with the initial node.
2. Expand the most promising node using heuristic values.
3. Identify whether the node is an **AND node** or **OR node**.
4. For an **OR node**, select the minimum-cost alternative.
5. For an **AND node**, include all required successor nodes.
6. Update the cost values from child nodes toward the root.
7. Mark nodes as **solved** when their required solution paths are completed.
8. Continue until the initial node is solved.

4. Simple Example



Suppose:

- A can be solved through **B or C**
- B requires **G and H**
- C can be solved through **E or F**

AO* evaluates the heuristic costs and selects the **minimum-cost solution graph**.

Key Difference

A* Algorithm	AO* Algorithm
Works on ordinary graphs	Works on AND-OR graphs
Finds an optimal path	Finds an optimal solution graph
Selects one path	May require multiple subproblems
Uses heuristic search	Uses heuristic search with AND-OR decomposition

Applications of AO*

- Game playing
- Artificial Intelligence planning
- Expert systems
- Problem reduction
- Automated theorem proving
- Diagnosis systems

In short: AO* is useful when a problem can be decomposed into **alternative choices (OR)** and **multiple required sub-problems (AND)**.

OR GAMES (Alternative Choices)

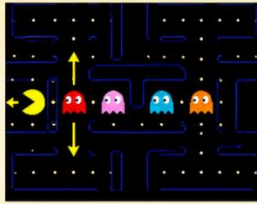
In OR games, the player can choose **ONE** of the alternative options.

CHESS



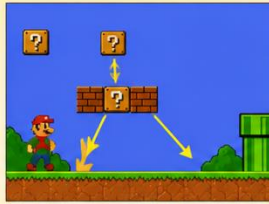
From one position, the player can move the knight to **ONE** of the possible squares.

PAC-MAN



Pac-Man can move in **ONE** of the possible directions (Up, Down, Left, or Right).

SUPER MARIO



Mario can choose to jump, go right, or enter the pipe – **ONE** option at a time.

AGE OF EMPIRES



The player can choose to build a Barracks **OR** a Stable **OR** a Siege Workshop.

AND GAMES (Multiple Requirements)

In AND games, the player must achieve **ALL** of the required subgoals.

STARCRAFT



To attack successfully, you need **MULTIPLE** things: Workers **AND** Army **AND** Resources.

WARCRAFT III



To win a battle, you need Heroes **AND** Units **AND** Upgrades.

DIABLO



To defeat a boss, you must have Strong Weapons **AND** Good Skills **AND** Potions.

MINECRAFT (Survival Mode)



To make a pickaxe, you need Wooden Planks **AND** Sticks. Both are required.

PROBLEM REDUCTION

Artificial Intelligence – UNIT II

1. Definition

Problem reduction is an Artificial Intelligence problem-solving technique in which a **complex problem is divided into smaller and simpler subproblems**. The original problem is solved by solving the required subproblems.

2. AND–OR Graph

Problem reduction is commonly represented using an **AND–OR graph**. The graph contains OR nodes and AND nodes.

2.1 OR Node

An **OR node** represents alternative ways of solving a problem. Solving **any one** of the successor nodes is sufficient.

Example:

$A \rightarrow B \text{ or } C$

Therefore, $A = B \text{ OR } C$

2.2 AND Node

An **AND node** represents a problem that must be divided into multiple subproblems. **All** successor nodes must be solved.

Example:

$A \rightarrow B \text{ and } C$

Therefore, $A = B \text{ AND } C$

3. Example of an AND–OR Graph

Suppose a problem A can be solved either by solving C, or by solving B. If solving B requires both D and E, then:

$A = (D \text{ AND } E) \text{ OR } C$

Thus, either C alone can solve A, or both D and E must be solved through B.

4. Applications of Problem Reduction

- Theorem proving
- Planning
- Expert systems
- Diagnosis problems
- Complex problem decomposition

5. Relation to AO* Algorithm

The **AO*** algorithm is used to search an AND–OR graph and determine a suitable solution graph for a problem-reduction task.

6. Summary

Problem Reduction = Breaking a large AI problem into smaller subproblems and solving them using an AND–OR structure.

Concept	Meaning
OR Node	Any one successor subproblem is sufficient
AND Node	All successor subproblems must be solved
AND–OR Graph	Represents both alternative and combined subproblems
AO*	Search algorithm used for AND–OR graphs

GAME PLAYING – ADVERSARIAL SEARCH

Artificial Intelligence – UNIT II

1. Introduction

Game playing is an important application of Artificial Intelligence in which two or more players make decisions to achieve their objectives. In many AI games, one player's gain is the other player's loss.

2. Adversarial Search

Adversarial search is a search technique used when an intelligent agent must select an action while considering the possible actions of an opponent. Unlike ordinary search problems, the environment contains competing or opposing agents.

Examples

- Chess
- Tic-Tac-Toe
- Checkers
- Connect Four

3. Players in Adversarial Search

MAX Player: Tries to obtain the maximum possible score and selects the best move for itself.

MIN Player: Tries to minimize the score obtained by MAX and reduce MAX's advantage.

4. Game Tree

A **game tree** represents the possible moves and outcomes of a game.

```
MAX
/\
MIN MIN
/\ /\
3 5 2 9
```

Evaluation:

Left MIN node = $\min(3, 5) = 3$

Right MIN node = $\min(2, 9) = 2$

Root MAX node = $\max(3, 2) = 3$

Therefore, **MAX selects the left branch.**

5. Important Components of a Game

1. **Initial State** – Starting position of the game.
2. **Players** – Agents participating in the game.
3. **Actions** – Possible moves available to a player.
4. **Result Function** – Defines the state produced by an action.
5. **Terminal Test** – Checks whether the game has ended.
6. **Utility Function** – Assigns a numerical value to the final state.

6. Minimax Principle

The **Minimax principle** is used in adversarial search. MAX chooses the move with the **highest value**, while MIN chooses the move with the **lowest value**.

7. Minimax Algorithm

MINIMAX(node)

If node is a terminal node

 return utility value

If node belongs to MAX

 return maximum value among its children

If node belongs to MIN

 return minimum value among its children

8. Advantages

- Helps an AI agent select the best possible move.
- Considers the opponent's possible actions.
- Useful for two-player competitive games.
- Forms the basis for intelligent game-playing systems.

9. Applications

- Chess
- Tic-Tac-Toe
- Checkers
- Connect Four
- Other strategic two-player games

10. Summary

Game Playing – Adversarial Search is an AI search process in which an agent selects the best action while assuming that the opponent will also try to select the best action against it. The **Minimax algorithm** is commonly used for such games.

Concept	Meaning
Adversarial Search	Search in the presence of an opponent
MAX	Player trying to maximize the score
MIN	Player trying to minimize MAX's score
Game Tree	Representation of possible game moves and outcomes
Minimax	Algorithm based on maximizing and minimizing values

1. What is a Game in AI?

In **Artificial Intelligence (AI)**, a **game** is a problem in which **two or more players compete with each other by making a sequence of moves**.

Each player has a **goal**. Usually:

- One player tries to **win**.
- The other player tries to **prevent the first player from winning**.
- Players choose their moves based on the current situation of the game.

AI can use algorithms such as **Mini-Max** to determine the **best possible move**.

Examples of Games in AI

Game	Players	AI Objective
Chess	2	Checkmate the opponent
Tic-Tac-Toe	2	Get three symbols in a row
Checkers	2	Capture/block opponent pieces
Connect Four	2	Connect four symbols
Othello	2	Have more pieces at the end

2. Important Terms in Game Playing

A. Game State

A **game state** represents the **current situation or position of the game**.

For example, in Tic-Tac-Toe:

X | O | X

| X |

O | |

This complete board represents the **current game state**.

So:

Game State = Current position of all players and game elements.

B. Initial State

The **initial state** is the **starting position of the game before any move is made**.

For example, an empty Tic-Tac-Toe board:

| |

| |

| |

This is the **initial state**.

So:

Initial State = Starting position of the game.

C. Move / Action

A **move** or **action** is a possible decision that a player can make during the game.

For example, in Tic-Tac-Toe:

```
| |  
-----  
| X |  
-----  
| |
```

Placing X in the center is a **move**.

So:

Move = An action taken by a player to change the current game state.

D. Player

A **player** is the person or AI that makes a move.

There can be:

- Human vs Human
- Human vs AI
- AI vs AI

In Mini-Max, the two players are generally represented as:

MAX

MIN

3. MAX Player

MAX represents the player who wants to get the **highest possible score**.

MAX can be:

- The AI player
- The computer
- The player we are designing the algorithm for

For example:

Move A $\rightarrow$ 3

Move B $\rightarrow$ 5

Move C $\rightarrow$ 2

MAX chooses:

$\text{MAX}(3, 5, 2) = 5$

Therefore:

MAX always tries to maximize the score.

Easy memory trick:

MAX $\rightarrow$ Maximum $\rightarrow$ Highest value

4. MIN Player

MIN represents the opponent of **MAX**.

MIN wants to get the **lowest possible score for MAX**.

For example:

Move A $\rightarrow$ 3

Move B $\rightarrow$ 5

Move C $\rightarrow$ 2

MIN chooses:

$$\text{MIN}(3, 5, 2) = 2$$

Therefore:

MIN always tries to minimize MAX's score.

Easy memory trick:

MIN $\rightarrow$ Minimum $\rightarrow$ Lowest value

5. Terminal State

A **terminal state** is a state where the **game has ended**.

The game can end because:

- MAX wins
- MIN wins

- The game is a draw

For Tic-Tac-Toe:

```

X | X | X
-----
O | O | 
-----
| | 

```

MAX has three Xs in a row.

Therefore, this is a **terminal state**.

So:

Terminal State = Final state where no more moves are required.

6. Utility / Payoff

The **utility**, also called **payoff**, is a numerical value assigned to a terminal state to indicate whether the result is good or bad for MAX.

A simple scoring system is:

Game Result	Utility
MAX wins	+1
Draw	0
MIN wins	-1

For example:

MAX wins



Score = +1

Draw



Score = 0

MIN wins

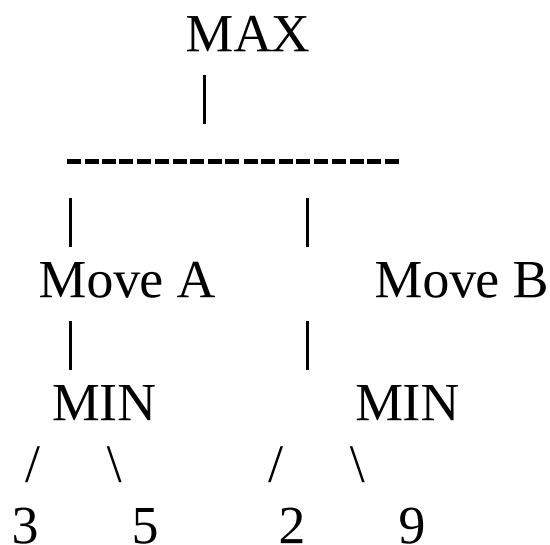


Score = -1

The Mini-Max algorithm uses these values to decide which move is best.

7. Complete Example

Consider this simple game tree:



Step 1 – MIN evaluates Move A

MIN sees:

3 and 5

MIN chooses the smaller value:

$$\text{MIN}(3,5) = 3$$

So:

$$\text{Move A} = 3$$

Step 2 – MIN evaluates Move B

MIN sees:

2 and 9

MIN chooses:

$$\text{MIN}(2,9) = 2$$

So:

$$\text{Move B} = 2$$

Step 3 – MAX chooses

MAX now sees:

$$\text{Move A} = 3$$

$$\text{Move B} = 2$$

MAX chooses:

$$\text{MAX}(3,2) = 3$$

Therefore:

MAX chooses Move A.



8. Summary Table

Term	Clear Meaning	Example
Game	Competition where players make moves	Chess
Game State	Current situation of the game	Current chess board
Initial State	Starting position	Empty Tic-Tac-Toe board
Move / Action	A player's possible action	Place X in a square
Player	Person or AI making a move	Human or computer
MAX	Player trying to get the highest score	AI
MIN	Opponent trying to reduce MAX's score	Opponent
Terminal State	Final state of the game	Someone wins/draw
Utility / Payoff	Numerical value of final result	+1, 0, -1



Simple Minimax Program

```
def minimax(depth, node, maximizing):  
    # Leaf node: return its value  
    if depth == 0:  
        return node  
  
    if maximizing:  
        return max(  
            minimax(depth - 1, node[0], False),  
            minimax(depth - 1, node[1], False)  
        )  
    else:  
        return min(  
            minimax(depth - 1, node[0], True),  
            minimax(depth - 1, node[1], True)  
        )  
  
# Game tree leaf values  
tree = [[3, 5], [2, 9]]  
result = minimax(2, tree, True)  
print("Best value for MAX:", result)
```

Output:

Best value for MAX: 3

Program

def minimax(depth, node, maximizing):

- Defines a function called minimax.
- depth → tells how many levels are left in the game tree.
- node → contains the current game-tree values.
- maximizing → tells whether it is MAX's turn (True) or MIN's turn (False).

if depth == 0:

return node

- Checks whether we have reached the **leaf/terminal node**.
- If depth becomes 0, the function returns the value of that node.

if maximizing:

- Checks whether the current player is **MAX**.
- MAX tries to select the **largest value**.

**return max(
 minimax(depth - 1, node[0], False),
 minimax(depth - 1, node[1], False)
)**

- MAX examines both possible moves: node[0] and node[1].
- depth - 1 moves one level down.
- False means that after MAX moves, it becomes **MIN's turn**.
- max() selects the **larger value**.

else:

- This executes when it is **MIN's turn**.

```

return min(
    minimax(depth - 1, node[0], True),
    minimax(depth - 1, node[1], True)
)

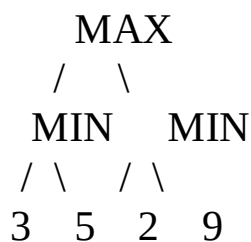
```

- MIN examines both possible moves.
- True means that after MIN moves, it becomes **MAX's turn**.
- min() selects the **smaller value** because MIN tries to minimize MAX's score.

Game Tree

```
tree = [[3, 5], [2, 9]]
```

This represents:



```
result = minimax(2, tree, True)
```

- Starts the Minimax algorithm.
- 2 → tree has 2 levels.
- tree → the game tree.
- True → **MAX plays first**.

MIN chooses:

- $\min(3, 5) = 3$
- $\min(2, 9) = 2$

Then MAX chooses:

- $\max(3, 2) = 3$

```
print("Best value for MAX:", result)
```

- Displays the final value selected by MAX.

Output

Best value for MAX: 3

In short:

MAX → chooses maximum → MIN → chooses minimum → finally MAX gets the best guaranteed value = 3.

Packages Useful for AI and Games

Package	Used For
NumPy	Mathematical calculations
Scikit-learn	Machine Learning algorithms
TensorFlow / PyTorch	Deep Learning
NLTK / spaCy	NLP
OpenCV	Computer Vision
Pygame	Game development
Turtle	Beginner graphics and simple games

Installing a package:

```
pip install pygame
```

For example, to install NumPy:

```
pip install numpy
```

Command	Purpose	Example
<code>pip install package</code>	Install a package	<code>pip install pygame</code>
<code>pip uninstall package</code>	Remove a package	<code>pip uninstall pygame</code>
<code>pip list</code>	Show installed packages	<code>pip list</code>
<code>pip show package</code>	Show package information	<code>pip show pygame</code>
<code>pip install --upgrade package</code>	Upgrade a package	<code>pip install --upgrade pygame</code>
<code>pip freeze</code>	Show installed packages and versions	<code>pip freeze</code>
<code>pip --version</code>	Check PIP version	<code>pip --version</code>

Python Concept	Use in Games
Variables	Store player position, score, health
<code>if-else</code>	Game decisions and conditions
<code>for</code> loop	Repeat actions
<code>while</code> loop	Main game loop
Lists	Store multiple enemies or objects
Functions	Create reusable game actions
Input	Keyboard/mouse controls
Operators	Calculate movement and scores
Libraries such as Pygame	Graphics, sound, keyboard input, and game windows